

AI Agent Orchestration Platform

Problem Statement

Build a platform where users can create AI agents, configure how they behave and operate (personality, tools, schedules, memory, limits), and connect them into collaborative workflows.

Agents must run on a real runtime, execute real tools, and communicate with each other to complete tasks autonomously. At least one agent must be reachable through an external messaging channel (WhatsApp, Telegram, or Slack) so a human can interact with it conversationally.

The platform must include a web UI for managing everything visually.

Business Context

The candidate must deliver a working repository with:

- A README explaining architecture decisions
- How to run the project
- A recorded demo (video or gif) showing the end-to-end workflow in action — including a live conversation with an agent through the chosen messaging channel

A live demo session will be scheduled to walk through the code and discuss tradeoffs.

Evaluation Weights

Criteria	Weight
Working end-to-end demo	40%
Architecture and code quality	30%
UI/UX and configurability	20%
Documentation	10%

Key Impact Metrics

- Number of configurable dimensions per agent
- Time from zero to a working multi-agent workflow
- End-to-end task completion rate
- Agent-to-agent message reliability

Technical Requirements

Programming Languages

Candidate's choice — must justify the decision in the README.

AI/ML Frameworks

Must integrate one of these agent runtimes as the execution engine:

OpenClaw (<https://openclaw.ai>) — Always-on personal agent framework. Agents defined via markdown files (SOUL.md, MEMORY.md). Built-in scheduler, memory system, and multi-channel support.

OpenCode (<https://opencode.ai>) — Terminal-native coding agent. Strong at code generation, file manipulation, and shell execution. Lightweight and fast.

Goose (<https://block.github.io/goose/>) — Open-source AI agent by Block. Extension-based tool system, session management, and provider-agnostic model support.

Candidate must choose one and explain the tradeoff in the README.

Development Tools

Candidate's choice for frontend and backend stack. Must include:

- A web-based UI
- A persistence layer (for agent configs, memory, workflow state, and execution history)
- Real-time communication between frontend and backend (WebSocket or SSE)

Cloud Platforms

Optional — the project must run fully local with a single setup command.

Agent Communication Requirements

Agents must communicate asynchronously:

1. When Agent A finishes work, it sends a message to Agent B with the output
2. Agent B picks it up, processes it, and either passes it forward or sends it back with feedback
3. The platform must persist the message history so the UI can show the full conversation trail between agents

At least one agent must be connected to an external messaging channel (WhatsApp, Telegram, or Slack) — a user must be able to open that channel, send a message to the agent, and get a real response. The demo must show this working.

The chosen runtime must actually execute the agent logic — this is not a UI mockup exercise.

Functional Requirements (Must-Haves)

Agent CRUD

Create, edit, delete agents from the UI. Each agent has:

- Name, Role, System prompt, Model, Tool access, Communication channels

Agent Configuration

For each agent, configure:

- **Schedules** — Cron jobs or intervals that wake the agent to perform tasks
- **Memory** — Persistent facts and preferences the agent retains across sessions
- **Skills** — Reusable step-by-step procedures combining multiple tools
- **Interaction rules** — What the agent can do autonomously vs. what requires approval
- **Guardrails** — Cost limits, rate limits, blocked actions

Workflow Builder

Connect agents into workflows visually. Define execution order, conditions, and feedback loops.

Example: A 'Dev Environment' workflow where:

1. A Coder agent writes code
2. A Reviewer agent reviews it
3. If rejected → sends feedback back to Coder
4. If approved → a Deployer agent deploys it

This loop must be configurable, not hardcoded.

Workflow Templates

At least 2 pre-built templates that users can load and modify (e.g., development pipeline, research pipeline).

External Channel Integration

At least one agent must be accessible through WhatsApp, Telegram, or Slack:

- The user must be able to chat with the agent from the messaging app
- The agent must respond using its configured personality, tools, and memory
- The UI should show which channel each agent is connected to

Live Monitoring

Real-time view of:

- Agent status, Logs, Inter-agent messages (including messages received from external channels)
- Task progress, Basic token/cost tracking

Working End-to-End Demo

At least one workflow with 2+ agents must execute a real task:

- Agents must actually call tools, produce output, exchange messages, and reach a conclusion
- Additionally, demo a human chatting with one of the agents through the connected messaging channel

Code Quality Expectations

- Clear separation between UI layer, agent runtime integration, and data/persistence layer
- Tests for critical paths (agent creation, workflow execution, message delivery)
- README with: Architecture diagram, Setup instructions, Runtime choice justification, Instructions for adding a new workflow template or a new messaging channel

Performance Benchmarks

N/A — the focus is on functionality and architecture, not on hitting specific numbers.

It should feel responsive and work smoothly.